Each microservice owns its own database (database-per-service pattern) using SQL Server. Services never share tables directly — all cross-service data sync happens via Kafka events. The entity lists below are extracted from the service specifications; field-level types, keys, and relationships were not specified in the source document and should be finalised with the engineering team during implementation.

1. Auth Service — Database

Owns: Login credentials, tokens, PIN hash, verification state.

Key Entities

- User Credential (email, password hash, PIN hash)
- Refresh Token
- Email Verification OTP
- Failed Login Attempt Counter

2. User Service — Database

Owns: Profile data, roles, KYC status.

Key Entities

- User Profile
- Role Assignment
- KYC Status Record

3. Wallet Service — Database ★ Financial System of Record

Owns: All balances and the full ledger. This is the most critical database in the system — every other service's financial state is downstream of it.

Key Entities

- Wallet (MainBalance, BudgetBalance per user)
- WalletLedger (before/after balances, debit type, IdempotencyKey — full audit trail of every money movement)
- Idempotency Key Record (prevents double-processing of financial operations)

Design Notes:

- Every ledger entry must record the balance before and after the operation, plus which balance (Main / Budget) was affected — this is what makes wallet.budget.debited vs wallet.main.debited routing possible downstream.
- IdempotencyKey must be unique and checked before any write to prevent duplicate credits/debits from retried requests or webhook re-deliveries.
- Pessimistic locking is applied at the row level during debit operations to prevent race conditions (e.g. two simultaneous vendor payments overdrawing a budget).

4. Budget Service — Database

Owns: Budget plans, daily allocation tracking, gift budget records.

Key Entities

- Budget (total amount, daily amount, start/end date, Status: Pending/Active/Completed/Cancelled/Failed)
- Daily Budget Tracker (UserTotalDailyBudget, spent-today amount, date)

- Gift Budget Record (sender, receiver, amount, cancellable flag = false)

Design Notes:

- Daily spend tracking uses FIFO consumption against the day's released amount.
- Gift budgets are explicitly flagged as non-cancellable at the schema level to prevent accidental reversal logic being applied to them.

5. Transfer Service — Database ★ NEW

Owns: External bank transfer records and their lifecycle status.

Key Entities

- Transfer (sender UserId, amount, destination bank code, destination account number, recipient name, Status: Pending/Completed/Failed, Paystack transfer reference)
- Bank Account Verification Cache (account number, bank code, resolved account name)

Design Notes:

- Status transitions (Pending → Completed / Failed) must be driven only by verified Paystack webhook events, never by client-side confirmation.
- A scheduled reconciliation job should poll Paystack for any Transfer left in Pending beyond a timeout window (24h) and trigger the failure/reversal path.

6. Payment Service — Database

Owns: Deposit records from Paystack virtual account webhooks.

Key Entities

- Deposit Record (Paystack reference, amount, userId, Status: Successful/Failed, raw webhook payload for audit)

7. Vendor Pay Service — Database

Owns: In-platform payment requests (vendor QR / user-to-user).

Key Entities

- Payment Request (requester UserId, payer UserId, amount, method: QR/UserId, Status: Requested/Approved/Rejected/Expired, expiry timestamp — 2 minute window)

8. Transaction Service — Database

Owns: The cross-service transaction lifecycle ledger — the coordinator's own record of every financial operation in flight or completed.

Key Entities

- Transaction (type: InPlatformPayment/Deposit/ExternalTransfer/GiftBudget, Status: Pending/Processing/Completed/Failed, related entity references, timestamps)

9. Notification Service — Database

Owns: Notification delivery records for tracking and retry.

Key Entities

- Notification (userId, channel: Push/SMS/Email, template used, delivery Status, retry count)

10. Cross-Service Data Flow Summary

Since each service owns its own database, the following table shows which services need read-access (via events, not direct DB access) to data owned elsewhere:

Data	Owning Service	Consumed By (via Kafka)
User identity	Auth Service	User Service
User profile	User Service	Vendor Pay Service, Notification Service
Wallet balances	Wallet Service	Budget Service, Transaction Service, Transfer Service, Notification Service
Budget status	Budget Service	Wallet Service, Notification Service
Transfer status	Transfer Service	Wallet Service, Transaction Service, Notification Service
Deposit status	Payment Service	Wallet Service, Transaction Service, Notification Service
Payment request status	Vendor Pay Service	Wallet Service, Transaction Service
Transaction status	Transaction Service	Vendor Pay Service, Notification Service